

# UartProtocol 串口协议文件格式规范

`.UartProtocol` 是基于 JSON 语法的配置文件，用于向串口协议分析器（如 Oscilloscope 的 UART 模块）提供目标设备的帧结构、通信波特率、固定的报文头（Header），以及各条指令的发送（Tx）和接收（Rx）报文格式及字段解码映射。

挂载 `.UartProtocol` 文件后，分析器可直接在图形界面中对截获的 UART 字节流进行自动断帧分包，并对每帧数据进行字节（Byte）级别的解析和释义翻译。

## 1. 根级别结构 (Root Structure)

每个 `.UartProtocol` 文件的顶层必须是一个合法的 JSON 对象，包含以下基本字段：

```
{
  "name": "OAS-1000D100",
  "description": "OAS-1000D100 OpticSensor UART Protocol",
  "baudRate": 115200,
  "header": ["0x55", "0xAA"],
  "commands": {
    ...
  }
}
```

| 字段名                      | 类型                 | 说明                                                                                                 |
|--------------------------|--------------------|----------------------------------------------------------------------------------------------------|
| <code>name</code>        | String (必填)        | 设备名称或协议名称，用于在 UI 的挂载列表和标题栏中展示。                                                                     |
| <code>description</code> | String (选填)        | 协议的简要说明或备注信息。                                                                                      |
| <code>baudRate</code>    | Int (选填)           | 默认的串口波特率（如 <code>115200</code> 、 <code>9600</code> 等），可帮助分析器进行波特率预设。                               |
| <code>header</code>      | Array[String] (必填) | 报文帧头（Frame Header）定义。是一个十六进制字符串的数组。分析器在处理字节流时，将依靠此特征字节序列来进行断帧同步。例如 <code>["0x55", "0xAA"]</code> 。 |

| 字段名                   | 类型          | 说明                                                                                |
|-----------------------|-------------|-----------------------------------------------------------------------------------|
| <code>commands</code> | Object (必填) | 协议指令映射字典。其键 (Key) 为指令命令码 (CMD, 如 <code>"0x01"</code> ) , 其值 (Value) 为对应指令的收发详细定义。 |

## 2. 命令定义 (Command Definition)

`commands` 字典中的每一个 Key 应该是一个表示十六进制指令码的字符串 (如 `"0x01"`, `"0x70"`) , Value 为该指令的属性对象, 定义了这根指令的发送报文 (`tx`) 和接收报文 (`rx`) 规范。

```
"0x01": {
  "name": "Baud Rate Setting",
  "tx": {
    "length": 8,
    "payload": [ ... ]
  },
  "rx": {
    "length": 8,
    "payload": [ ... ]
  }
}
```

| 字段名               | 类型          | 说明                                                |
|-------------------|-------------|---------------------------------------------------|
| <code>name</code> | String (必填) | 该指令的操作名称或简写 (如 <code>Baud Rate Setting</code> ) 。 |
| <code>tx</code>   | Object (选填) | 该指令的主机发送帧 (Transmit Frame) 格式定义。如果没有发送帧可省略。       |
| <code>rx</code>   | Object (选填) | 该指令的从机应答/接收帧 (Receive Frame) 格式定义。如果没有应答可省略。      |

## 3. 报文帧结构 (Frame Definition: tx / rx)

无论是 `tx` 还是 `rx` , 都遵循相同的报文帧结构定义:

```
"tx": {
  "length": 8,
  "payload": [
    ...
  ]
}
```

| 字段名     | 类型                 | 说明                                            |
|---------|--------------------|-----------------------------------------------|
| length  | Int (必填)           | 该报文帧的总字节数（包含 Header 在内）。解析器会根据帧头和此长度来提取完整的一帧。 |
| payload | Array[Object] (必填) | 包含该帧数据中各个有效字节（Byte）定义的数组。                     |

## 4. 字节字段定义 (Payload Field Definition)

`payload` 数组规定了该报文帧内（除帧头外）不同字节偏移处的数据含义。

```
{
  "byteOffset": 6,
  "name": "Type",
  "valueMap": {
    "0x01": "9600bps",
    "0x07": "115200bps"
  },
  "description": "Baud rate type"
}
```

| 字段名        | 类型          | 说明                                                                                          |
|------------|-------------|---------------------------------------------------------------------------------------------|
| byteOffset | Int (必填)    | 该字段所在的字节偏移量（从 0 开始计数）。例如，如果帧头有 2 个字节，那么数据的 <code>byteOffset</code> 通常从 2 开始。                |
| name       | String (必填) | 该字节的字段缩写或名字（如 <code>CMD</code> , <code>STA</code> , <code>Data1</code> ）。                   |
| value      | String (选填) | 固定期望值。如果填了该项（如 <code>"0x01"</code> ），分析器可校验该位置的值是否与之匹配（通常用于 CMD 字节或保留位 <code>0xFF</code> ）。 |

| 字段名                      | 类型             | 说明                                                                                             |
|--------------------------|----------------|------------------------------------------------------------------------------------------------|
| <code>valueMap</code>    | Object<br>(选填) | <b>字段值映射表</b> 。键 (Key) 为十六进制字符串 (如 <code>"0x01"</code> )，值 (Value) 是对应的释义。解析器会根据读取的字节值直接翻译出文本。 |
| <code>description</code> | String (选填)    | 该字节的详细解释 (如 <code>"SUM[3:7]"</code> 等校验和说明)。                                                   |

## 5. 编写建议

- 固定头偏移**：既然已经在根目录定义了 `header` 数组，在 `payload` 数组中就**无需**再次为 `byteOffset: 0` 和 `byteOffset: 1` 编写说明，可以直接从数据的实际起始偏移 (通常为 `2`) 开始编写。
- 校验和 (Checksum) 处理**：通常可以将报文的最后一个字节命名为 `"Checksum"`，在 `description` 中标注它的计算方式 (如 `SUM[1:7]` 代表累加和)，方便后续查阅。